

uvg266: Open-Source VVC Intra Encoder

Marko Viitanen
Tampere University
Tampere, Finland
marko.viitanen@tuni.fi

Joose Sainio
Tampere University
Tampere, Finland
joose.sainio@tuni.fi

Kari Siivonen
Tampere University
Tampere, Finland
kari.siivonen@tuni.fi

Alexandre Mercat
Tampere University
Tampere, Finland
alexandre.mercat@tuni.fi

Jarno Vanne
Tampere University
Tampere, Finland
jarno.vanne@tuni.fi

Abstract

Versatile Video Coding (VVC/H.266) standard is the emerging successor to the widespread High Efficiency Video Coding (HEVC/H.265). This work introduces the latest version of our academic open-source VVC intra encoder called uvg266. It has been developed from our well-known Kvazaar HEVC encoder by introducing new VVC coding tools into carefully optimized and parallelized coding flow of Kvazaar. This paper outlines the design methodology and implementation aspects of all intra (AI) configuration of uvg266. The experimental results show that single-threaded uvg266 is more than twice as fast as the state-of-the-art VVenC encoder in all our test cases. In speed-optimized coding, the coding overhead of uvg266 is 21.7% but the gap narrows down to 2.6% in the rate distortion optimized coding case. uvg266 has almost linear speedup with core count up to 32 threads. The better scalability of uvg266 quadruples the speed over VVenC. Furthermore, single-threaded uvg266 is up to 380× as fast as VVC reference software VTM and the gap raises to over 11,000× with 32 threads. To the best of our knowledge, uvg266 is currently the fastest available open-source VVC intra software encoder.

CCS Concepts

• Software and its engineering • Computing methodologies ~ Computer graphics ~ Image compression

Keywords

Open source, Versatile Video Coding (VVC), VVC intra coding, uvg266 VVC encoder

ACM Reference format:

Marko Viitanen, Joose Sainio, Kari Siivonen, Alexandre Mercat and Jarno Vanne. 2024. uvg266: Open-Source VVC Intra Encoder. In *Proceedings of the 32nd ACM International Conference on Multimedia (MM '24)*, October 28–November 1, 2024, Melbourne, VIC, Australia. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3664647.3685525>

1 Introduction

The demand for efficient video compression continues to grow with the proliferation of video applications and more immersive media content. *Versatile Video Coding (VVC/H.266)* [1] doubles the coding efficiency over that of *High Efficiency Video Coding (HEVC/H.265)* [2] for the same subjective visual quality. However, the coding gain of VVC comes with increased encoding complexity, posing a challenge for developers seeking for practical and real-time VVC encoding solutions [3] [4].

Currently, there exist three prominent open-source VVC encoders: 1) *VVC Test Model (VTM)* [5], which is the reference encoder with all normative VVC tools but the coding speed is far from practical; 2) *Fraunhofer Versatile Video Encoder (VVenC)* [6], which is currently the most popular practical VVC encoder; and 3) uvg266 [7], which is presented in this work. VVenC was optimized from VTM, which can result in performance trade-offs due to bloated structure and legacy of standardization.

This paper presents an overview of uvg266, an open-source software VVC encoder, that has been developed from our open-source Kvazaar HEVC encoder. Our previous work [7] introduced the first steps of uvg266 development, whereas this work presents the first version of uvg266 with a complete toolset for VVC intra coding. The 3-clause BSD licensed source code is available online in GitHub at

<https://github.com/ultravideo/uvg266/>,

encouraging further community-driven development and exploration. Even though part of the inter coding tools are also implemented into uvg266, they are not yet optimized for VVC and are thereby considered out of scope of this paper.

This paper is structured as follows. Section 2 gives an overview of the uvg266 encoder. Section 3 reports the intra coding efficiency and speed of uvg266 and compares the results with the other open-source encoders. Section 4 gives instructions on how to deploy the encoder. Finally, Section 5 concludes the paper.



This work is licensed under a Creative Commons Attribution International 4.0 License.

MM '24, October 28–November 1, 2024, Melbourne, VIC, Australia.

© 2024 Copyright is held by the owner/author(s).

ACM ISBN 979-8-4007-0686-8/24/10. <https://doi.org/10.1145/3664647.3685525>

2 uvg266 VVC Encoder

2.1 From HEVC to VVC

The primary research question behind the development of uvg266 is how to effectively convert an existing, practical, and heavily optimized Kvazaar HEVC software encoder into a fully functional VVC encoder. To that end, uvg266 has been designed to inherit the core principles of Kvazaar, which include cross-platform compatibility, openness, and efficiency for real-time coding.

The transition from HEVC to VVC within uvg266 was started by updating existing Kvazaar coding tools to comply with VVC specifications. This included modifications to CABAC contexts, block structures, and algorithms as detailed in our previous work [7]. The entire change history can be traced through as Git commits.

Overall, uvg266 inherits the general structure of Kvazaar. uvg266 can be used both as a standalone command line tool and as a library. The encoder control operates on parallelization units: threads, slices, and wavefront rows. For each parallelization unit, a single *coding tree unit (CTU)* or a smaller coding block can be processed at a time. Therefore, the thread queue splits work into CTUs and keeps track of the dependencies, maximizing the CPU utilization. Finally, the deployment of AVX2 optimized kernels makes the most of each CPU core.

2.2 Implementation of New VVC Coding Tools

Overall, uvg266 implements all VVC intra tools, as specified in Table 1. The implementation of ALF, LMCS, JCCR, and MTS was extensively covered in our previous work [7]. For other tools, the intrusiveness of the required changes varies greatly. Notably, DQ was among the simplest to integrate; once the tool was incorporated, it only requires updating the function calls, from regular quantization or RDOQ to new DQ function.

However, one of the most significant adaptations from HEVC to VVC is the introduction of non-square CUs facilitated by the MTT coding structure. This change represents a major modification at the top level, affecting numerous aspects of the encoding process including recursive search, intra prediction, transforms, quantization, cost calculation, deblocking, and bitstream generation. Additionally, the dual tree tool impacts the coded structure but involves less modifications. Since luma and chroma mode searches are already separated in the architecture, the primary alteration involves performing the recursive CU structure search twice, initially for luma and subsequently for chroma. Similar duplications are required in the bitstream generation to accommodate this structural change.

The implementation of new luma intra modes, MRL, MIP, and ISP, introduces distinct modifications. MRL adapts existing prediction functions by offsetting references based on the line count and necessitates updates to the reference pixel collection function and mode integration in the search algorithms. MIP, a new prediction mode, requires its addition to the search process but does not alter other aspects of the encoder structure. ISP divides traditional angular, DC, and planar modes into multiple prediction units, necessitating adaptations for non-square CUs as

Table 1: Overview of implemented coding tools in uvg266.

Feature	Details
VVC profiles	♦ Supports the Main10 profile.
Bit depth	♦ Supports 8-bit 4:2:0 video.
Coding structures	♦ Implements <i>quad tree+multi-type tree (QT+MTT)</i> structure with <i>coding unit (CU)</i> sizes ranging from 64×64 to 4×4.
Intra prediction	♦ DC, Planar, and 65 angular prediction modes. ♦ <i>Position dependent intra prediction combination (PDPC)</i> . ♦ <i>Multi reference line (MRL)</i> , <i>matrix intra prediction (MIP)</i> , <i>intra subpartition (ISP)</i> , and <i>wide angle intra prediction (WAIP)</i> . ♦ <i>Cross component linear model (CCLM)</i>
Transforms	♦ <i>Multiple transform selection (MTS)</i> with implicit and explicit modes. ♦ DCT2, DST7, and DCT8 transforms. ♦ <i>Low frequency non-separable transform (LFNST)</i> . ♦ <i>Joint coding of chroma residuals (JCCR)</i>
Quantization	♦ Supports <i>quantization parameters (QP)</i> ranging from 0 to 63. ♦ <i>Rate distortion optimized quantization (RDOQ)</i> ♦ <i>Dependent quantization (DQ)</i> .
Loop filtering	♦ Deblocking filter with 4×4 grid. ♦ <i>Sample adaptive offset (SAO)</i> . ♦ <i>Adaptive loop filter (ALF)</i> . ♦ <i>Luma mapping with chroma scaling (LMCS)</i> .
Parallelization	♦ Supports multithreading using tiles and <i>Wavefront parallel processing (WPP)</i> .

facilitated by MTT. Similarly, the new chroma prediction mode CCLM uses cached downsampled luma pixels to avoid redundant recalculations, also requiring integration into the chroma mode search.

Finally, the LFNST itself is just an extra step between the transform and quantization and inverse transform and inverse quantization, with strict conditions when the LFNST mode is written to the bitstream.

2.3 Parallelization and Vectorization

Adapting the parallelization schemes from Kvazaar to meet the specific requirements of VVC in uvg266 included modifications like altering the order of CTU processing in WPP. In terms of vectorization, almost all AVX2 kernels used in Kvazaar have been retained and only required minor adjustments for expanded applications: the DCT2 transform kernel has been adapted for the DST7 and DCT8 transforms, and the intra prediction kernel originally used for luma in HEVC is now also effectively utilized for chroma prediction in VVC. Additionally, the same kernels are leveraged for cost calculation and quantization.

Additional AVX2 kernels has been proposed in uvg266 to address the introduction of non-square CUs in MTT [8], enabling also ISP optimizations in the future. Moreover, uvg266 benefits from novel optimization and AVX2 kernels for DQ [9].

3 Performance evaluation

3.1 Experimental Setup

Our experiments were carried out under AI VVC *common test conditions* (CTC) [10]. The evaluation was restricted to 8-bit input videos since Kvazaar and uvg266 are only optimized for that bit depth. Table 2 lists our test sequences that included 5 CTC sequences from classes A1/A2 and 16 sequences from the UVG dataset [11].

uvg266 was benchmarked against three other encoders: 1) VTM, 2) VVenC, and 3) Kvazaar, with versions and source codes detailed in Table 3. Each encoder is distributed under a permissive BSD-3-clause license. VTM serves as the VVC reference software. It establishes the baseline for interpreting the *Bjontegaard delta bitrate* (BD-rate) results. VVenC represents the state-of-the-art in practical VVC encoding. Kvazaar provides a comparative basis between HEVC and VVC, sharing its codebase with uvg266.

Table 4 lists the encoding parameters used in the evaluation. For uvg266 and Kvazaar, the presets called *ultrafast* and *veryslow* were selected, whereas *faster* and *slower* were used with VVenC. VTM lacks any coding speed specific presets, so it was evaluated using the default AI preset. Table 5 specifies our profiling platform.

3.2 Single-threaded Performance

Tables 6 and 7 provides the single-threaded benchmarking results in terms of PSNR BD-rate [12], [13] and speedup, respectively. Results show that uvg266 *veryslow* is 9.4× as fast as VTM, but with an expected BD-rate increase of 16.5%. In *ultrafast*, uvg266 is on average 382× as fast as VTM with 63.9% increase in BD-rate.

Compared against VVenC, uvg266 is 2.3× and 2.2× as fast with an averaged BD-rate increase of +21.7% and +2.6% in *ultrafast* and *veryslow* presets, respectively. The gap is smaller in the *veryslow* case, partly because uvg266 works better with the noisy background of *Beauty* and *Lips* than VVenC.

uvg266 outperforms Kvazaar by -2.7% and -14.4% BD-rate with a 0.3× and 0.1× speedup in *ultrafast* and *veryslow* presets, respectively. With *ultrafast* preset, this is due to the updated tools (more intra angles, PDPC) in VVC. For *veryslow* preset, the difference is more noticeable since multiple new VVC tools are enabled, slowing the process down but also providing a significant BD-rate gain.

3.3 Multi-threaded Performance

Table 8 presents the speedups achieved by each practical encoder when using 8, 16, and 32 threads. Ideally, these speedups would scale linearly with a thread count, but practical constraints like synchronization overhead prevent this. Our results show that VVenC cannot take full advantage of all the cores because its threading is not optimally configured for the AI coding.

Kvazaar and uvg266 share the same threading implementation that effectively utilize nearly all 32 threads on a 32-core machine. Their small performance difference can be attributed to the higher frame parallelism of VVC in WPP processing. In VVC, a new CTU

Table 2: Test sequences.

Class	Resolution	Name(s)	Frames
A1	3840×2160	Campfire	300
		FoodMarket4	720
		Tango	294
A2	3840×2160	CatRobot, DaylightRoad2	300
		ParkRunning3	300
UVG	3840×2160	CityAlley, FlowerFocus, Lips, FlowerKids, FlowerPan, RaceNight, RiverBank, Twilight, Beauty, Bosphorus, HoneyBee, Jockey, ReadySetGo, YachtRide	600
		ShakeNDry, SunBath	300

Table 3: Encoder versions.

Name	Version	Available
uvg266	0.8.1	https://github.com/ultravideo/uvg266
VTM	23.3	https://vcgit.hhi.fraunhofer.de/jvet/VVCSoftware_VTM
VVenC	1.11.1	https://github.com/fraunhoferhhi/vvenc
Kvazaar	2.3.1	https://github.com/ultravideo/kvazaar

Table 4: Encoding parameters.

Encoder	Option
uvg266	--preset=<preset> -p1 --threads=<threads>
VTM	-c encoder_intra_vtm.cfg
VVenC	-c randomaccess_<faster slower>.cfg --IntraPeriod=1 --IntraQPOffset=0 --Threads=<threads>
Kvazaar	--preset=<preset> -p1 --threads=<threads>

Table 5: Profiling platform.

Processor	AMD ThreadRipper 2990WX (32 × 4.2 GHz)
Architecture	x86 /AVX2
Memory	64 GB
Compiler	GCC 11.4.0
Operating Syst.	Ubuntu 22.04

row can be started after processing only one CTU in the previous row, whereas two CTUs are required in HEVC.

With 32 threads, uvg266 can encode 4K intra frames at 12.7 fps in *ultrafast* preset. Using the data from Tables 7 and 8 we can calculate that encoding a 2-hour 4k 60Hz video would take VTM more than 12 years, while VVenC would do it in 40 hours, and uvg266 would take just 9.4 hours.

4 Software Deployment

uvg266 is a self-contained software that includes a pthread wrapper for Windows, eliminating the need for additional libraries. CMake [14] is used to generate build files for Windows, Linux, and Mac. Assuming a recent gcc or clang compiler is available, compiling on Linux involves generating build files with **cmake**, and then compiling the software using **make**. If no errors occur, the **uvg266** is compiled and ready for use.

The simplest command to compress a YUV 4:2:0 format raw input video is **./uvg266 -i in.yuv --input-res=1280x720 -o out.vvc**. The resulting file is in the raw VVC format, which can be decompressed using tools like the VTM decoder. Support in *FFmpeg* [15] and other software for live playback is still in progress.

Table 6: BD-rate (PSNR) comparison between uvg266, VTM, VVenC, and Kvazaar.

Encoder	Anchor →	uv266		VTM	VVenC		Kvazaar	
	Preset ↓→	ultrafast	veryslow	default	faster	slower	ultrafast	veryslow
uv266	ultrafast	0.0%	39.5%	63.9%	21.7%	43.8%	-2.7%	19.2%
	veryslow	-28.1%	0.0%	16.5%	-12.9%	2.6%	-30.1%	-14.4%
VTM	default	-38.7%	-13.9%	0.0%	-27.8%	-15.1%	-40.2%	-26.7%
VVenC	faster	-17.4%	16.2%	39.8%	0.0%	18.1%	-20.0%	-0.8%
	slower	-30.0%	-0.8%	19.1%	-15.2%	0.0%	-32.1%	-15.3%
Kvazaar	ultrafast	2.8%	43.5%	68.2%	25.6%	48.4%	0.0%	22.8%
	veryslow	-15.8%	16.9%	36.9%	1.97%	20.0%	-18.3%	0.0%

Table 7: Single-threaded speedup comparison between uvg266, VTM, VVenC, and Kvazaar.

Encoder	Anchor →	uv266		VTM	VVenC		Kvazaar	
	Preset ↓→	ultrafast	veryslow	default	faster	slower	ultrafast	veryslow
uv266	ultrafast	1.0×	40.0×	382.1×	2.3×	89.9×	0.3×	3.2×
	veryslow	<0.1×	1.0×	9.4×	0.1×	2.2×	<0.1×	0.1×
VTM	default	<0.1×	0.1×	1.0×	<0.1×	0.2×	<0.1×	<0.1×
VVenC	faster	0.5×	18.1×	171.8×	1.0×	38.1×	0.1×	1.4×
	slower	<0.1×	0.6×	5.1×	<0.1×	1.0×	<0.1×	<0.1×
Kvazaar	ultrafast	3.1×	124.0×	1186.6×	6.9×	278.1×	1.0×	9.8×
	veryslow	0.3×	12.6×	118.7×	0.7×	27.4×	0.1×	1.0×

5 Conclusion

uv266 presents a viable approach to developing a practical VVC intra encoder by building it upon the established Kvazaar HEVC encoder framework. It is more than 2× as fast as the state-of-the-art VVenC encoder with a moderate coding overhead. Being open source, uv266 not only demonstrates the feasibility of converting an existing HEVC encoder to an efficient VVC software encoder but it also offers the academia and industry a platform for further development and exploration of VVC technology.

Future work will focus on making uv266 a full-fledged and powerful VVC encoder. These activities will include integrating and balancing VVC inter tools for better BD-rate, optimizing and reducing complexity through advanced implementation techniques such as machine learning, extending support to higher bit depths and various chroma subsampling modes, and refining presets to incorporate these new VVC coding tools effectively.

Acknowledgements

This work was supported in part by the AI-based Situational Awareness (AISA) and XR Simulation and Presence at the Cloud Edge (XR-SPACE) projects led by Nokia and funded by Business Finland, and the Research Council of Finland (#349216). The authors would also like to thank all contributors of uv266.

References

- [1] ITU, “New ‘Versatile Video Coding’ standard to enable next-generation video compression,” Sept. 2020, [Online]. Available: <https://www.itu.int/en/mediacentre/Pages/pr13-2020-New-Versatile-Video-coding-standard-video-compression.aspx>.
- [2] G. J. Sullivan, J. -R. Ohm, W. -J. Han, and T. Wiegand, “Overview of the high efficiency video coding (HEVC) standard,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 22, no. 12, pp. 1649–1668, Dec. 2012.
- [3] A. Mercat, A. Mäkinen, J. Sainio, A. Lemmetti, M. Viitanen, and J. Vanne, “Comparative rate-distortion-complexity analysis of VVC and HEVC video codecs,” *IEEE Access*, vol. 9, pp. 67813–67828, May 2021.

Table 8: Comparison of threading speedup.

Threads	uv266		VVenC		Kvazaar	
	ultrafast	veryslow	faster	slower	ultrafast	veryslow
8	7.8×	8.0×	7.0×	7.5×	7.5×	7.8×
16	15.4×	15.9×	11.9×	13.7×	14.6×	15.4×
32	30.3×	31.5×	16.2×	17.2×	27.2×	29.6×

- [4] J. Sainio, A. Mercat, and J. Vanne, “Design space exploration of practical VVC encoding for emerging media applications,” *IEEE Trans. Consum. Electron.*, vol. 68, no. 4, pp. 387–400, Nov. 2022.
- [5] “VVC Reference Software Version 13.2,” [Online]. Available: https://vcgit.hhi.fraunhofer.de/jvet/VVCSoftware_VTM/-/tree/VTM-13.2.
- [6] A. Wiecekowsky *et al.*, “Vvenc: an open and optimized vvc encoder implementation,” in *Proc. IEEE Int. Conf. Multimedia Expo. Workshops*, Shenzhen, China, Jul. 2021.
- [7] M. Viitanen, J. Sainio, A. Mercat, A. Lemmetti, and J. Vanne, “From HEVC to VVC: the first development steps of a practical intra video encoder,” *IEEE Trans. Consum. Electron.*, vol. 68, no. 2, pp. 139–148, Jan. 2022.
- [8] K. Siivonen, J. Sainio, A. Mercat, and J. Vanne, “Tailored AVX2 transform kernels for Versatile Video Coding,” in *Proc. IEEE Nordic Circuits Syst. Conf.*, Aalborg, Denmark, Oct.–Nov. 2023.
- [9] J. Sainio, A. Mercat, and J. Vanne, “Vectorized and optimized dependent quantization for practical VVC encoding,” in *Proc. IEEE Visual Comm. Image Proc.*, Jeju, Korea, Dec. 2023.
- [10] F. Bossen, J. Boyce, K. Suehring, X. Li, and V. Seregin, “VTM common test conditions and software reference configurations for SDR video,” *document JVT-T2010*, Teleconference, Oct. 2020.
- [11] A. Mercat, M. Viitanen, and J. Vanne, “UVG dataset: 50/120fps 4K sequences for video codec analysis and development,” in *Proc. ACM Multimedia Syst. Conf.*, Istanbul, Turkey, pp. 297–302, May 2020.
- [12] G. Bjøntegaard, “Improvements of the BD-PSNR model,” *document VCEG-AI11*, Berlin, Germany, Jul. 2008.
- [13] ITU-T and ISO/IEC JTC 1, “Working practices using objective metrics for evaluation of video coding efficiency experiments,” *document ITU-T HSTP-VID-WPOM and ISO/IEC DTR 23002-8*, 2020.
- [14] “CMake: A Powerful Software Build System,” [Online]. Available: <https://cmake.org/>.
- [15] “FFmpeg,” [Online]. Available: <https://ffmpeg.org>.